

Scientific Computing Report - EMAT 30008

Josh Beal - 2005289 - gx20774@bristol.ac.uk

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
from ODEs import solve_ode, RK4_step
from shooting import numshoot, plotorbit, general_pc
from plot import plotsystemode, plotphaseportrait
from NumericalContinuation import natpar, plot_continuation
from BVP import solve_poisson, plot_poisson
from PDEs import xgrid, maxdt, explicit_euler, implicit_euler, diffusionIC, plot
```

1. Software Demonstration

Question 1a

```
In [ ]: # Parameters
def brusselator(X, t, **params):    # Define ODE
    x, y = X
    A = params['A']
    B = params['B']

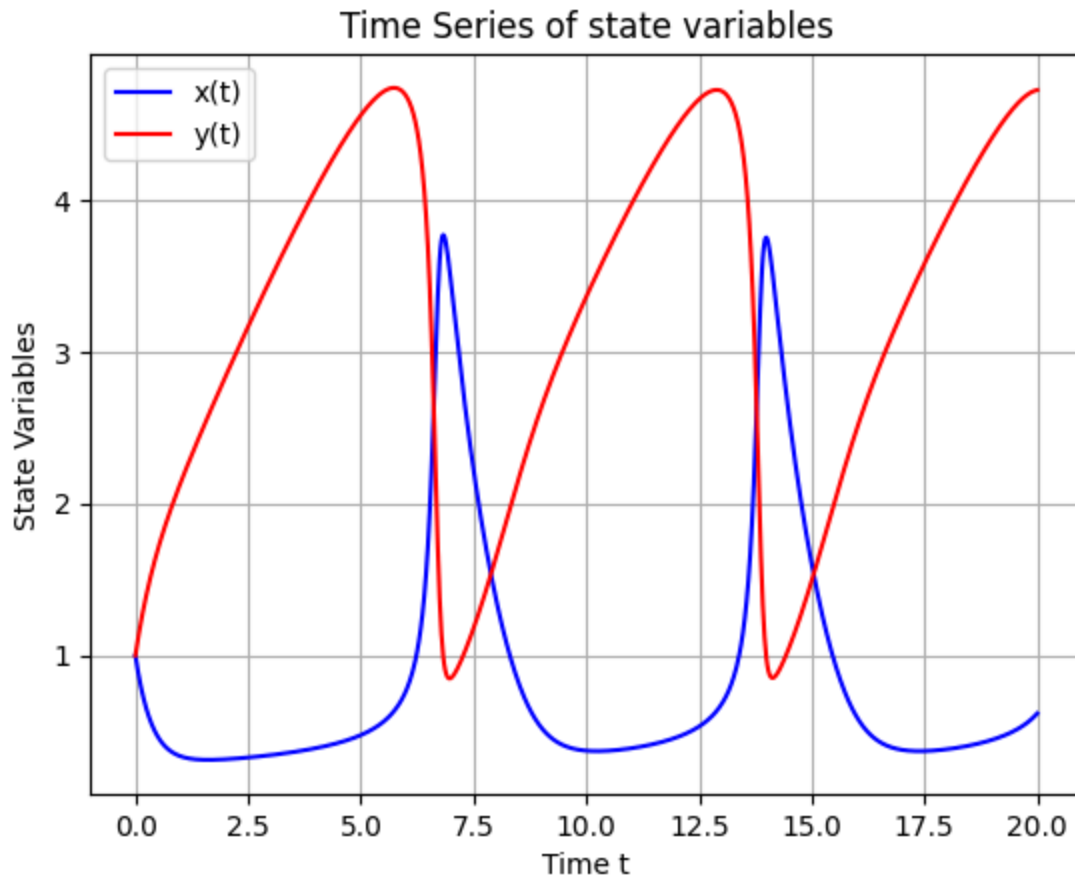
    dxdt = A + x**2 * y - (B + 1) * x
    dydt = B * x - x**2 * y

    X = [dxdt, dydt]
    return X

f = brusselator    # Choose ODE
X0 = [1, 1]        # Initial conditions
t = np.linspace(0, 20, 1000)    # Time vector
method = 'RK4'     # Integration method
A = 1              # Extra parameters
B = 3

X = solve_ode(f, X0, t, method, A=A, B=B)    # Solve ODE

plotsystemode(X, t)    # Plot ODE
```



Question 1b

```
In [ ]: # Parameters
def brusspc(X0, **params): # Define phase condition
    pbruss = brusselator(X0, 0, **params)[0]
    return pbruss

f = brusselator # Choose ODE
X0 = [1, 1] # Initial conditions
phasecondition = brusspc # Phase condition
Tguess = 8 # Initial guess for period
A = 1 # Extra parameters
B = 3 # Extra parameters

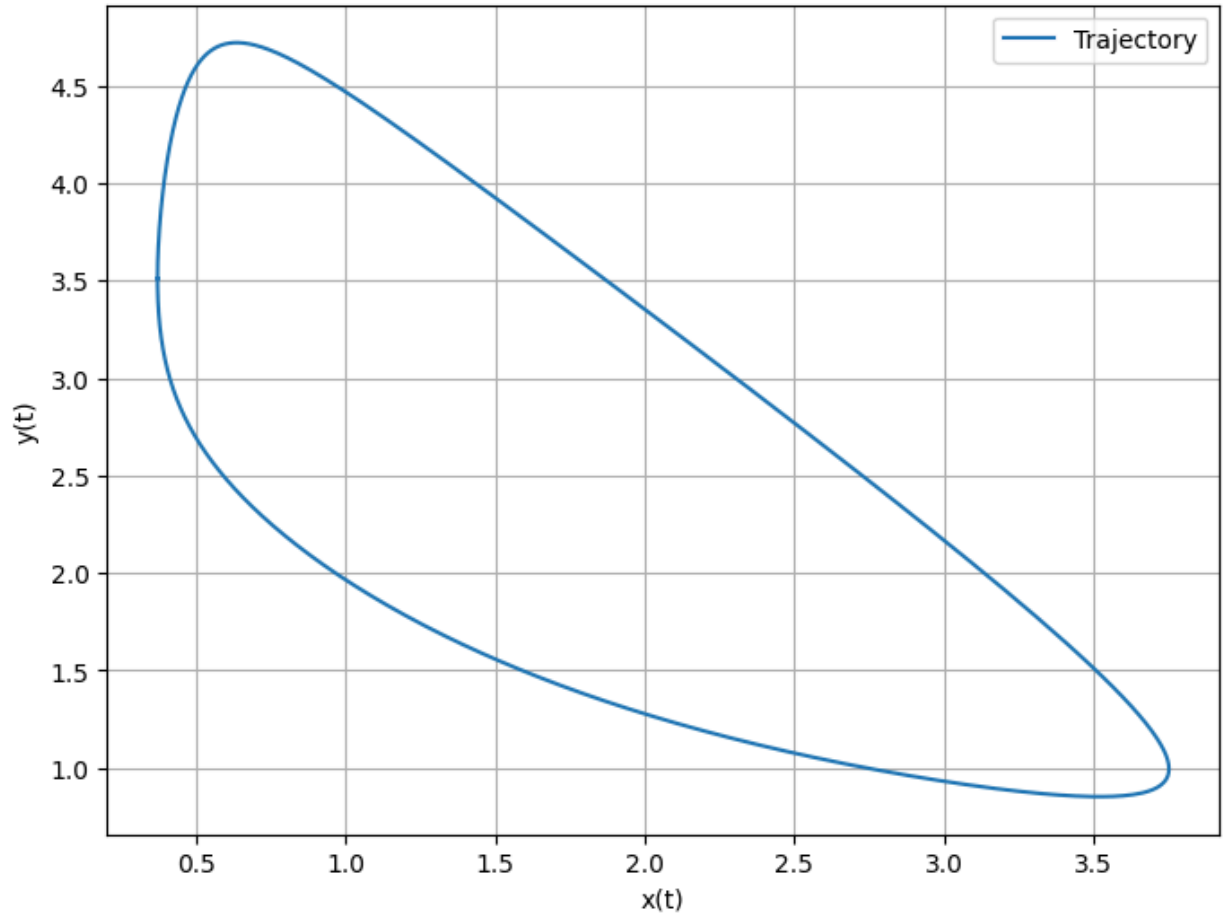
X0, T = numshoot(f, phasecondition, X0, Tguess, A=A, B=B)

x0, y0 = X0 # Initial conditions
t = np.linspace(0, T, 1000) # Time vector
X = solve_ode(f, X0, t, method, A=A, B=B) # Solve ODE

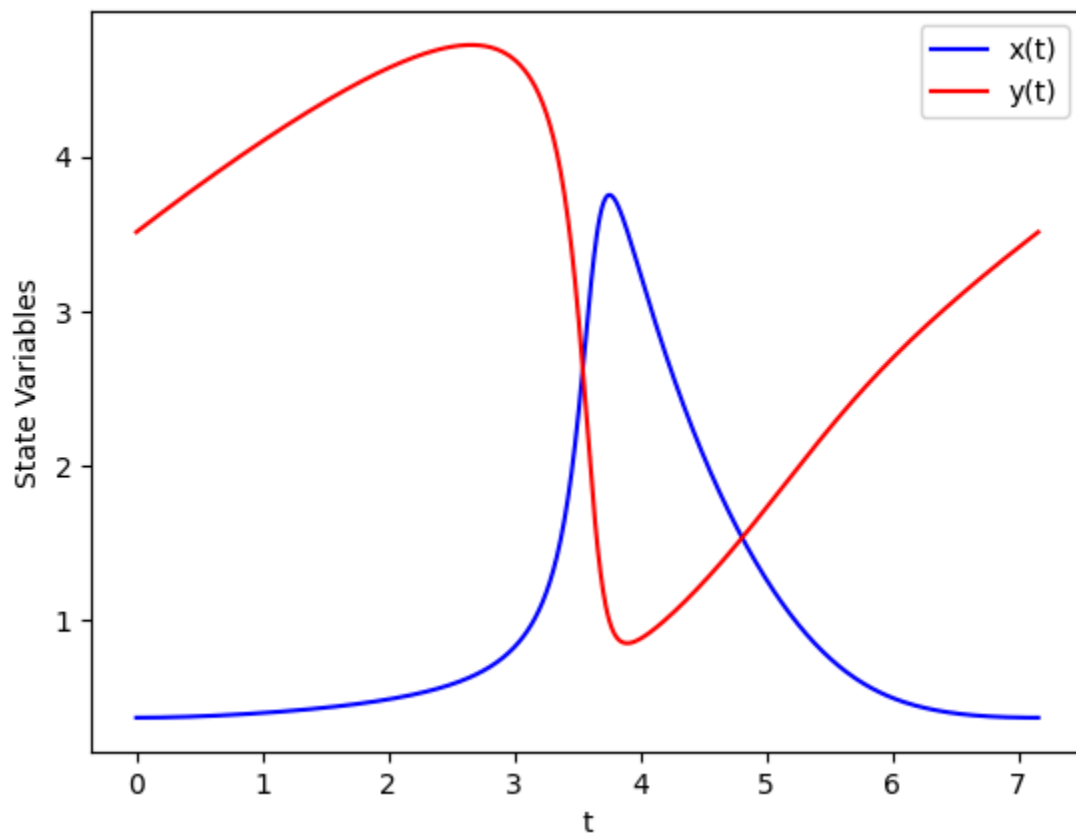
print(f"Optimized Initial Conditions: x(0) = {x0:.4f}, y(0) = {y0:.4f}") # /
print(f"Optimized Period: T = {T:.2f}") # Print optimized period
plotphaseportrait(X) # Plot phase portrait
plotorbit(f, X0, T, A=A, B=B) # Plot orbit

Optimized Initial Conditions: x(0) = 0.3706, y(0) = 3.5119
Optimized Period: T = 7.16
```

2D Phase Portrait



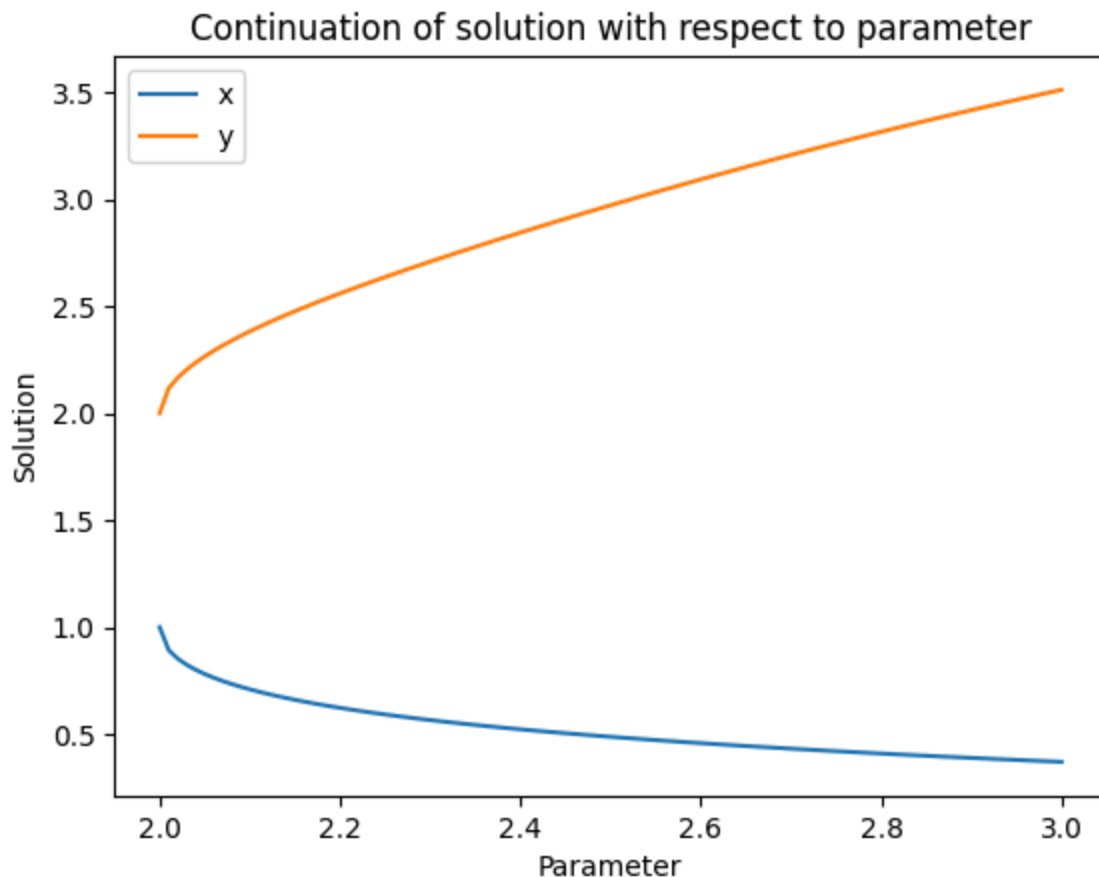
Time Series of State Variables



Question 1c

```
In [ ]: # Parameters
f = brusselator # Choose ODE
u0 = [0.4, 3.5] # Initial conditions
parvalues = (3, 2, 100) # Parameter values
parameter = 'B' # Parameter to vary
Tguess = 6.5 # Initial guess for period
phasecondition = brusspc # Phase condition
discretisation = 'shooting' # Discretisation method
A = 1 # Extra parameters

u0, parvals, Ts = natpar(f, [u0], parvalues, parameter, Tguess, phasecondition)
plot_continuation(u0, parvals) # Plot continuation
```



Question 2a

```
In [ ]: # Parameters
def hopf(X, t, **params): # Define ODE
    x, y, z = X
    mu = params['mu']
    dxdt = mu * x - y - z + x * (x**2 + y**2 + z**2) - x * (x**2 + y**2 + z**2)
    dydt = x + mu * y - z + y * (x**2 + y**2 + z**2) - y * (x**2 + y**2 + z**2)
    dzdt = x + y + mu * z + z * (x**2 + y**2 + z**2) - z * (x**2 + y**2 + z**2)
    X = [dxdt, dydt, dzdt]
    return X
f = hopf # Choose ODE
X0 = [1, 0, -1] # Initial conditions
```

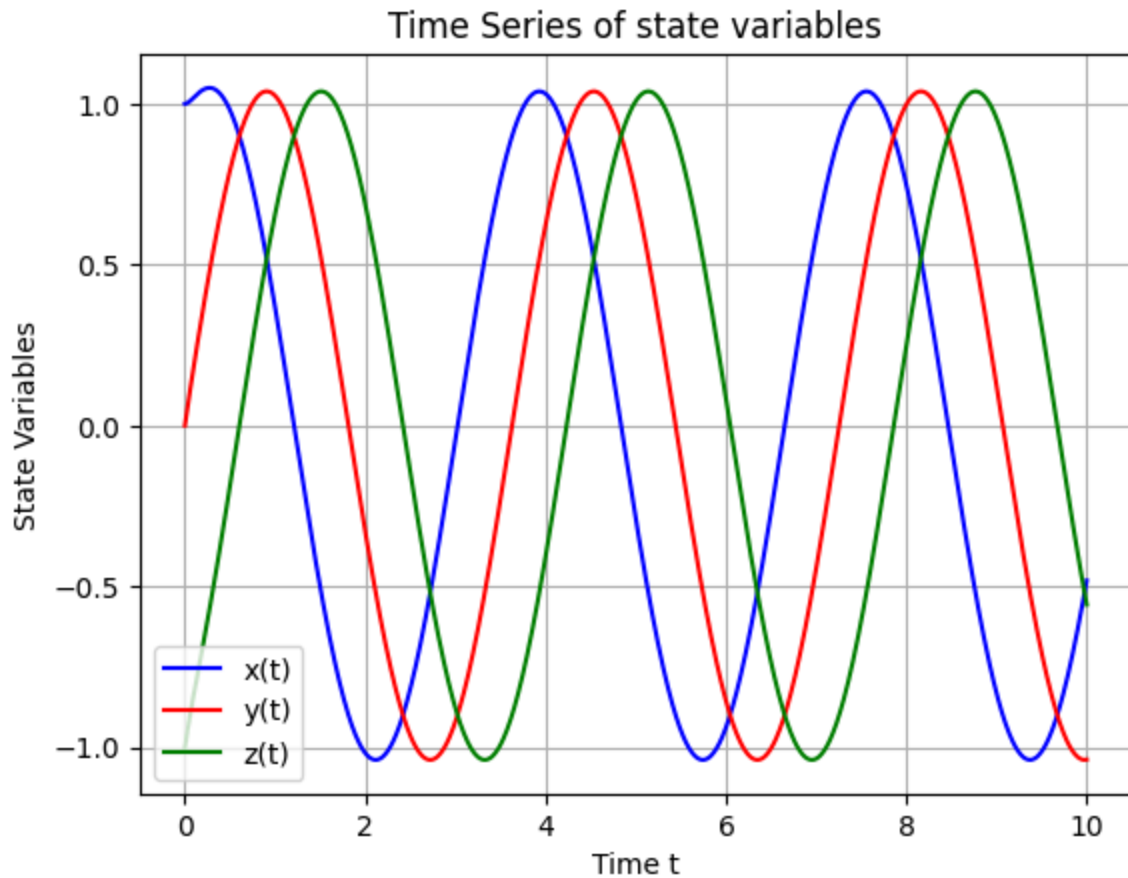
```

t = np.linspace(0, 10, 1000) # Time vector
method = 'RK4' # Integration method
mu = 1 # Extra parameters

X = solve_ode(f, X0, t, method, mu=mu) # Solve ODE

plotsystemode(X, t) # Plot ODE

```



Question 2b

```

In [ ]: # Parameters
def hopfpc(X0, **params): # Define phase condition
    pchopf = hopf(X0, 0, **params)[0]
    return pchopf
f = hopf # Choose ODE
X0 = [1, 0, -1] # Initial conditions
phasecondition = hopfpc # Phase condition
Tguess = 3 # Initial guess for period
mu = 1 # Extra parameters

X0, T = numshoot(f, phasecondition, X0, Tguess, mu=mu) # Shooting method

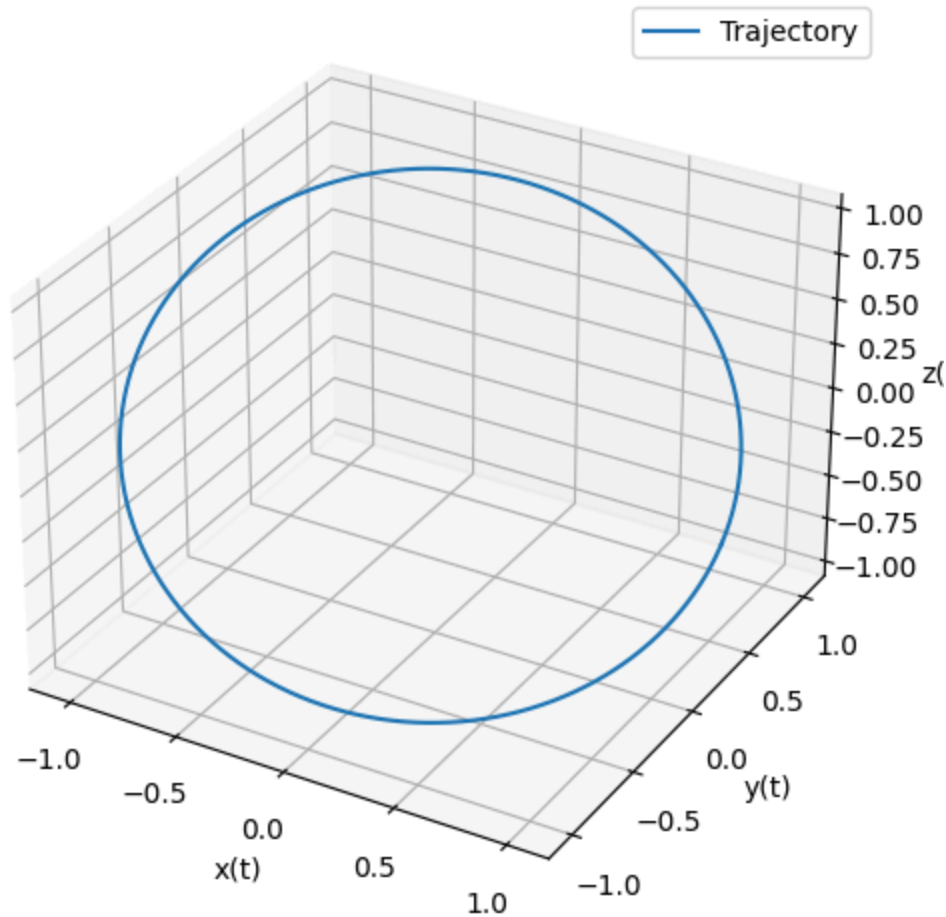
x0, y0, z0 = X0 # Unpacking the optimized initial conditions
period = T # Optimized period
t = np.linspace(0, period, 1000) # Time vector
X = solve_ode(f, X0, t, method, mu=mu) # Solve ODE

print(f"Optimized Initial Conditions: x(0) = {x0:.4f}, y(0) = {y0:.4f}", f"z(0) = {z0:.4f}")
print(f"Optimized Period: T = {period:.2f}") # Print optimized period
plotphaseportrait(X) # Plot phase portrait

```

Optimized Initial Conditions: $x(0) = 1.0332$, $y(0) = 0.5246$, $z(0) = -0.5246$
 Optimized Period: $T = 3.63$

3D Phase Portrait



Question 3a

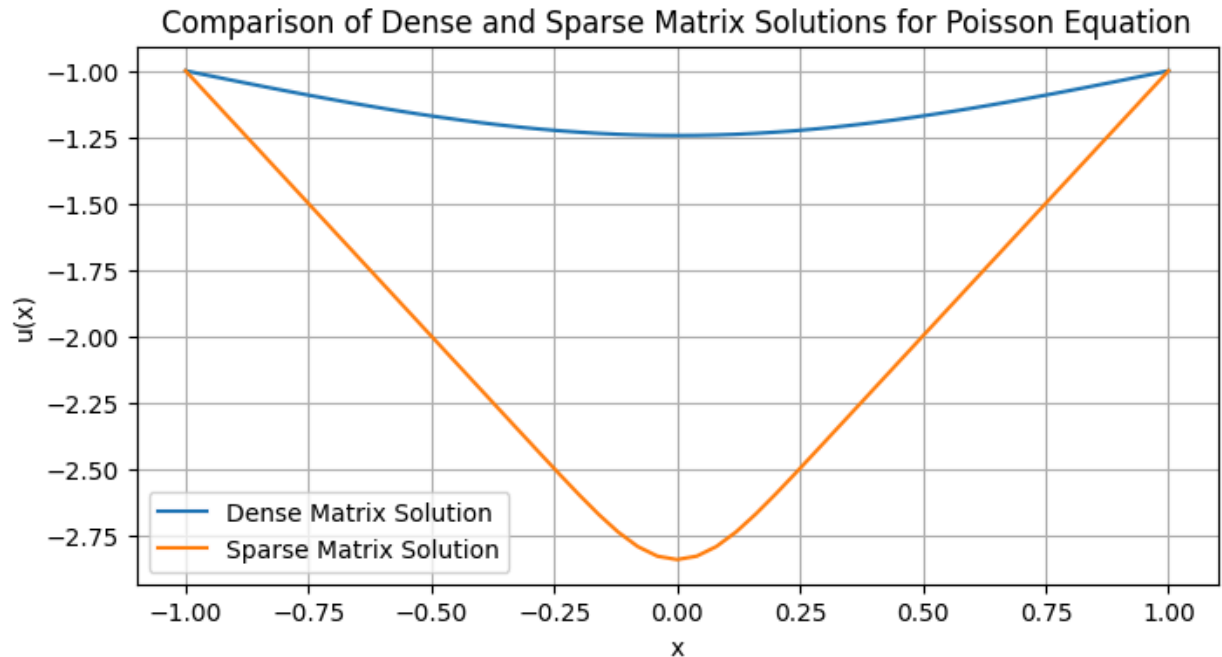
```
In [ ]: # Parameters

lower_x = -1    # Lower bound of x
upper_x = 1     # Upper bound of x
num_points = 51 # Number of points in x
D = 1          # Diffusion
bc_left = -1    # Boundary condition at x = lower_x
bc_right = -1   # Boundary condition at x = upper_x

sigma = 0.5     # Sigma
xdense, udense, tdense = solve_poisson(lower_x, upper_x, num_points, D, sigma,
sigma = 0.1     # Sigma
xsparse, usparse, tsparse = solve_poisson(lower_x, upper_x, num_points, D, sigma)

xlist = [xdense, xsparse] # List of x values
ulist = [udense, usparse] # List of u values
labels = ['Dense Matrix Solution', 'Sparse Matrix Solution'] # Labels for the plots
title = 'Comparison of Dense and Sparse Matrix Solutions for Poisson Equation'
plot_poisson(xlist, ulist, labels, title) # Plot Poisson equation solutions
```

Value of u at $u(0)$ (for $\sigma=0.5$) : -1.2432
 Time to solve using dense method: 0.0041 seconds
 Value of u at $u(0)$ (for $\sigma=0.1$) : -2.8377
 Time to solve using sparse method: 0.0013 seconds



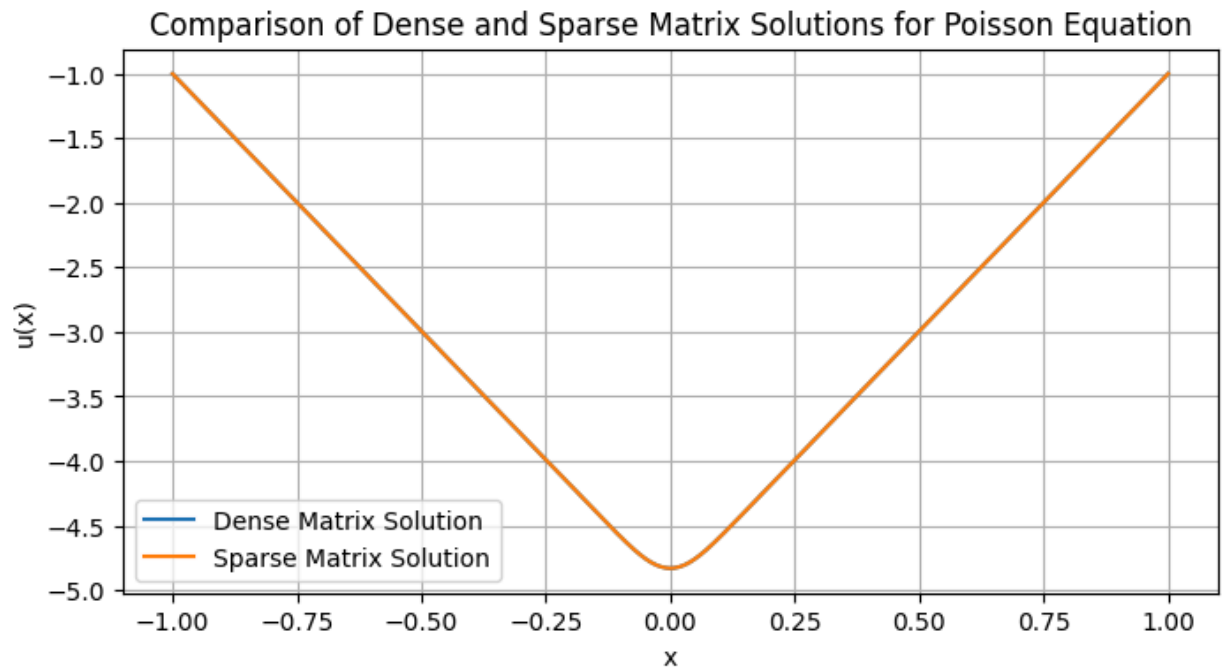
Question 3b

```
In [ ]: # Parameters
lower_x = -1
upper_x = 1
num_points = 501
D = 1 # Diffusion
bc_left = -1 # Boundary condition at x = lower_x
bc_right = -1 # Boundary condition at x = upper_x

sigma = 0.05
xdense, udense, tdense = solve_poisson(lower_x, upper_x, num_points, D, sigma,
xsparse, usparse, tsparse = solve_poisson(lower_x, upper_x, num_points, D, sigma)

xlist = [xdense, xsparse] # List of x values
ulist = [udense, usparse] # List of u values
labels = ['Dense Matrix Solution', 'Sparse Matrix Solution'] # Labels for the solutions
title = 'Comparison of Dense and Sparse Matrix Solutions for Poisson Equation'
plot_poisson(xlist, ulist, labels, title) # Plot Poisson equation solutions
```

Value of u at $u(0)$ (for $\sigma=0.05$) : -4.8304
 Time to solve using dense method: 0.0076 seconds
 Value of u at $u(0)$ (for $\sigma=0.05$) : -4.8304
 Time to solve using sparse method: 0.0018 seconds



The sparse method is much faster. This is because dense matrices are usually $O(n^3)$ for matrix inversion which can be computationally expensive as the size of the system grows. Sparse matrices skip unnecessary calculations, and therefore operate more efficiently. Using real world larger matrices, this gap in timing would be larger.

Question 4a

```
In [ ]: # Parameters
L = 2
a = 0 # Lower bound of x
b = L # Upper bound of x
N = 100 # Number of points in x
D = 0.5 # Diffusion

x, dx, xint = xgrid(N, a, b) # Grid
dtmax = maxdt(D, dx) # Maximum time step for stability
print(f"Maximum time step for stability: {dtmax:.4f}") # Print maximum time s
```

Maximum time step for stability: 0.0004

Question 4b

```
In [ ]: # Parameters
L = 2
a = 0 # Lower bound of x
b = L # Upper bound of x
N = 100 # Number of points in x
D = 0.5 # Diffusion

def diffusionIC(xint, b): # Initial condition
    return 0.5 * xint * (b - xint)

IC = diffusionIC # Initial condition
alpha = 1 # Boundary condition at x = a
```

```

beta = 0      # Boundary condition at x = b
T = 0.5      # Final time
dt = maxdt(D, dx)  # Time step

expu, expt = explicit_euler(N, D, alpha, beta, a, b, dt*0.5, dx, T, xint, IC)

print(f"Explicit method: u(0, T) = {expu[0, -1]:.4f}") # Print value of u(0,
expu1 = expu[1:,-1] # Extracting the values of u at the final time

impu, impt = implicit_euler(N, D, alpha, beta, a, b, dt*2, dx, T, xint, IC) #

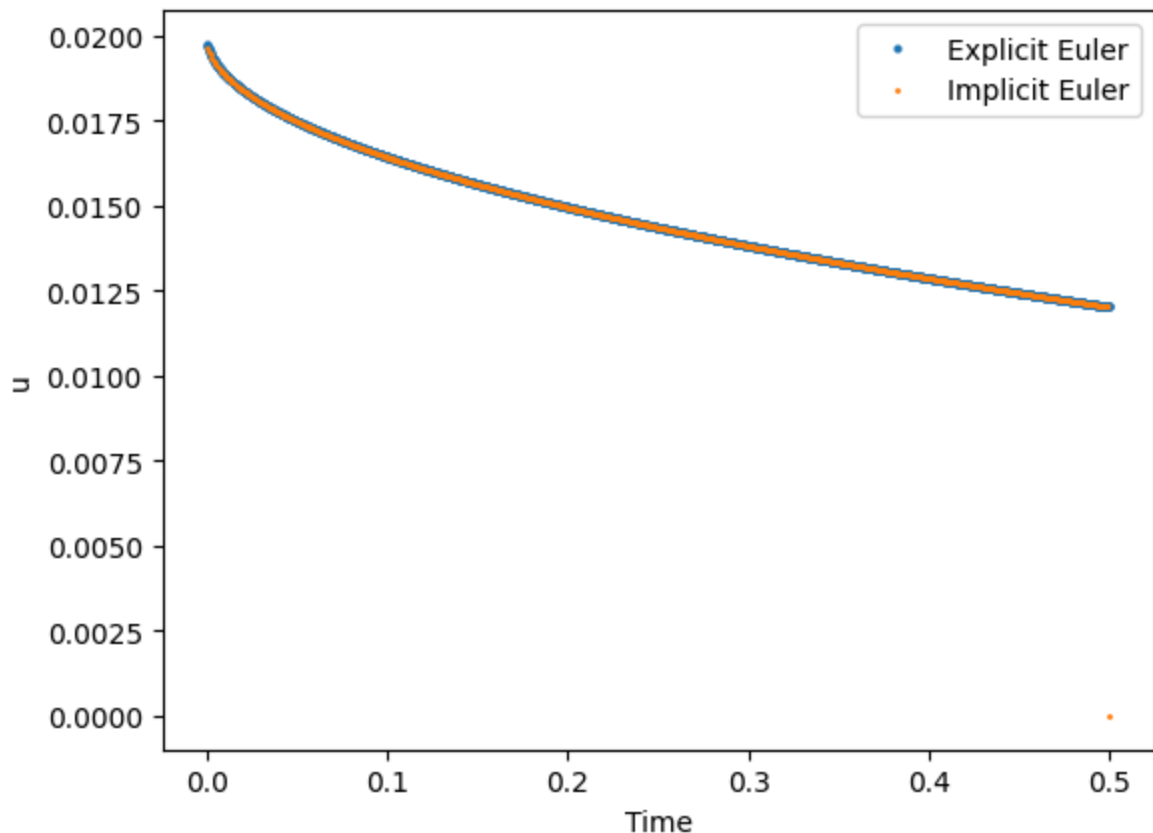
print(f"Implicit method: u(0, T) = {impu[0, -1]:.4f}") # Print value of u(0,
impu1 = impu[1:,-1] # Extracting the values of u at the final time

plot_pde(expu1, impu1, expt, impt)      # Plot PDE solutions

```

Explicit method: $u(0, T) = 0.0198$

Implicit method: $u(0, T) = 0.0198$



#

2. Description of the key software design decisions

2.1 ODEs

The `odes.py` module is structured to solve ordinary differential equations using multiple numerical methods. These include methods such as Euler, Runge-Kutta and Heun. The main function, `solve_ode`, uses a dictionary to map the names of each method so they can be

implemented. This allows for flexibility in selection through the simple interface. This also allows for very easy extensions of the code. Any step could be added into the function without changing the functionality of the code.

A decision I made for the `solve_to` function was adaptive step size. This means that as long as time to solve to is larger than the current value of t ($t < t_1$) it will perform the next iterative step. It then checks if the current t plus the time step is still smaller than t_1 ($t + dt < t_1$). If it is not smaller it calculates the difference ($dt = t_1 - t$) and will iterate once more before returning the final value.

I also considered using an object-oriented approach, to use each method within a class. Whilst this is very helpful for code organisation and simplifying the parameters, I chose the functional approach to simplify the usage. This also allows other contributors to make easy changes.

By checking the inputs to the `solve_ode` we can ensure the inputs passed by the user is the correct type to be handled. If it is not correct, it will raise an error such as unrecognised method. It then guides users to the correct type improving overall user experience.

2.2 Shooting

The `shooting.py` module is made for solving boundary value problems using the shooting method. This method transforms a boundary value problem into an initial value problem, which is then solved using the ODE solvers from `odes.py`. It also uses the `findroot` function I created, passing parameters through the `numshoot`, `findroot`, `solve_ode` functions all the way to the numerical methods. The use of the different modules demonstrates the cohesive design of the software. The `shooting.py` module has a lot of flexibility with parameters as they are passed through to the `odes.py` functions.

I chose to make the `Runge-Kutta` method the default solver within the shooting mechanism. I decided it had the best accuracy whilst not using excessive computational power. Whilst other methods could provide better stability for different types of problems, the `RK4` method has a wide range of applications.

I visualised the time series with a `plotsystemode` function. I also had another function to plot the phase portrait. A further enhancement of this would be to integrate the plot into the shooting function iterating over any number of variables and systems of ODEs. The module also incorporates error handling non-convergence making sure the results are reliable.

2.3 Continuation

The `NumericalContinuation.py` module is designed to explore solutions in parameterised systems, helping understand the bifurcations and stability in the mathematical models. This module implements natural parameter continuation and can navigate through complex solution landscapes. The continuation progresses through

parameter values in small customised increments, calculating the solutions at every step. This allows for tracing continuous solution branches and identifying where solutions may bifurcate or disappear.

One key part of this function is the `try: except` block so that errors are caught and located but the code will still run without crashing. Another important part of numerical continuation is the visualisation as this often gives us much more of an insight into what is really happening when changing the parameters. This is another instance where having an automatic plotting function can improve general user experience.

I have also attempted to implement pseudo arclength continuation into my module. Whilst I didn't get to fully develop the code, I tried to extend the parameter space by considering changes in both the solution and the parameter which can allow it to find a way through bifurcations or folds.

2.4 BVP

The `BVP.py` module uses the finite difference method to convert the boundary value problem into a system of equations that can be solved using numerical solvers. The continuous domain is discretised into a finite set of points whilst the differential equations are approximated using differences between these points. NumPy's `linalg` solver was used for a variety of reasons. Mainly because NumPy is optimised for numerical computations and has a significant speed advantage, especially for large matrices which are common for discretisation of differential equations for BVPs. Whilst being high performance, it is also very simple to use. This reduces the potential for bugs as it keeps the code simple.

The matrices are formed from the user inputs which can be specified to sparse or dense and each row approximates the differential equation at a specific point in the domain. Once the matrix and rhs vector are defined, the `linalg.solve` can be used to solve the dense linear system whilst `scipy.sparse.linalg`. We can then plot the results given to analyse and compare the dense and sparse matrix solutions.

2.5 PDE

The `PDEs.py` module uses numerical methods to solve the diffusion equation, specifically through the explicit and implicit Euler methods. It divides the domain between a and b by generating a spatial grid using the `xgrid` function. It is split into N equally spaced segments. We can use this to calculate the grid spacing (dx) and therefore the maximum stable step size (dt) using the Diffusion Coefficient (D). Both methods implemented solve the diffusion equation by discretising the spatial derivatives. As they are similar functions, I kept my code very similar so that in the future, a function can be created to generalise the PDE solvers. As I only implemented two methods to start with, I focused on them individually rather than making a module to do both at this stage. The explicit Euler method updates directly from previous time steps, while the implicit method involves

solving a system of linear equations using `linalg.solve` for stability at large time steps. The initial conditions are set through the `diffusionIC` function. Again, the best way to analyse the findings is by plotting and for the PDEs we use the `plot_pde` function.

Word Count : 1011

3. RLL

I started this project with little knowledge of numerical programming in python. In group projects, I found myself offering to do tasks that were not python based and if I needed to code I would use matlab. I have found this coursework and module extremely beneficial for my coding skills which have subsequently helped me for the rest of my course.

Exploring the algorithms for solving ODEs and PDEs has really helped me understand how they work and what they really mean. It has shown me the importance of each step in each method and the exponential effect it can have if a value or values have been miscalculated along the way. I hadn't seen natural parameter and pseudo arclength before but learning them in this way is much easier to retain because you are not memorising, you are understanding.

My mistake when I started was trying to do too much at once. I would make a function with as many parameters that I thought would be necessary and I would then work my way through with no real knowledge of what I needed to get as a result. I find it much easier now to start from the bottom. First, figure out what the question is asking or what I need to visualise as it is easy to change small snippets of small functions. Learning to modularise my code was something I thought would take years of practice so never thought I would. At the start I really struggled keeping track of the parameters, where they were going and where they were being input. I learned how to use key word arguments which I found much easier to track.

If I was to start again I would make sure to keep my tests. At the beginning, when testing I would test my code against a known solution and see if it was right and then delete it. I now know the importance of having a test file for nearly everything you can. Remaking tests because I made a small change in one of my functions was time consuming. This also leads onto my use of Git. I found it is much easier to restore to a previous commit as long as you commit regularly. I now constantly use Github for my group projects as it is easier to implement other people's functions whilst you are working in the same area rather than everyone writing their own different versions of the code.

When creating many different functions, I learned how useful it is to make sure that each function has a docstring. Looking back at code you made some time ago can feel like a different language. Using clear, concise docstrings which show inputs and outputs as well as their type, instantly reminds you of what the function does. They are also vital when working with other people as it allows them to use your functions easily.

This coursework has greatly improved my confidence in coding. I have learnt that in the future, when I am writing code, mistakes can be a good thing. I have thrown away files into the 'bin' folder to remind myself how many attempts it can take. However, the way you learn to fix the mistakes is much more important. The best way to improve my coding is to make sure I am adding tests along the way. The more descriptive the tests are, the more you learn from a mistake or a misinput. If a test or check can tell you exactly what is going wrong and where it greatly speeds up the process of debugging.

In summary, I did find this project very difficult and daunting at the start but I think what I have learnt will be very valuable for many future applications.

Word Count : 636